SHELL SCRIPTING

Table of Contents

Table of Contents 2
Chapter 1 — Introduction to Shell Scripting 8
What You Will Learn 8
How This Book Is Organized 8
Chapter 2 — Understanding the Shell 10
2.1 What Is a Shell? 10
2.2 Types of Shells 10
2.3 Checking Which Shell You Are Using 10
Check your distribution 10
Check your current logged-in user 11
Check your default shell 11
List all shells available on your system 11
Chapter 3 — What Is Shell Scripting? 12
3.1 Why Use Shell Scripts? 12
Chapter 4 — Writing and Running Your First Script 13
4.1 Setting Up a Working Folder 13
4.2 The Shebang Line 13
4.3 Naming Your Script 13
4.4 Creating the Script 13
4.5 Understanding echo 14
4.6 Running a Shell Script 14
Method 1 — Run the script using the bash command 14
Method 2 — Run the script directly with ./ 15
Method 3 — Run a script using its full path 15
Chapter 5 — Comments 16
5.1 Single-Line Comments 16
5.2 Multi-Line Comments 16
5.3 Why Comments Matter 16
Chapter 6 — Variables 17
6.1 What Is a Variable? 17
6.2 Declaring a Variable 17

6.3 Using (Printing) a Variable 17
6.4 Changing a Variable's Value 18
6.5 Storing a Command's Output in a Variable 18
6.6 Constant (Read-Only) Variables 18
Chapter 7 — Arrays 20
7.1 Why Use an Array? 20
7.2 Defining an Array 20
7.3 Accessing Array Elements 20
7.4 Retrieving All Values 20
7.5 Finding the Length of an Array 21
7.6 Retrieving a Range of Values 21
7.7 Updating (Appending to) an Array 21
7.8 Associative Arrays (Key-Value Pairs) 21
Chapter 8 — String Operations 23
8.1 Finding the Length of a String 23
8.2 Converting to Upper Case or Lower Case 23
8.3 Replacing Part of a String 23
8.4 Slicing (Extracting Part of) a String 24
Chapter 9 — Getting Input From the User 25
9.1 The read Command 25
9.2 Prompting the User Clearly 25
Method 1 — a separate echo statement 25
Method 2 — the -p flag of read (preferred) 25
Chapter 10 — Arithmetic Operations 27
10.1 Why You Can't Just Use * or + ... 27
10.2 Method 1 — the let Command 27
10.3 Method 2 — Double Parentheses $((...)) 28
Chapter 11 — Conditional Statements: if, elif, and case 29
11.1 The if Statement 29
Basic syntax 29
Worked example — pass or fail based on marks 29
11.2 Single vs. Double Square Brackets 29
11.3 Comparison Operators 30
11.4 The elif Statement — Handling Multiple Conditions 30
11.5 The case Statement (Switch Case) 31

Basic syntax 31
Worked example — a simple menu 32
Multiple commands inside one branch 32
Chapter 12 — Logical Operators 33
12.1 The AND Operator (&&) 33
12.2 The OR Operator (||) 34
12.3 Combining AND and OR 34
12.4 The Short-Form (Ternary-Style) Condition 34
Chapter 13 — Loops 35
13.1 The for Loop 35
Basic syntax 35
Example — looping over numbers 35
Example — looping over strings 35
Using a range instead of listing every value 35
13.2 for Loop With a File 36
13.3 for Loop With an Array 36
13.4 The while Loop 37
Basic syntax 37
Example — counting up to a limit 37
13.5 The until Loop 37
13.6 Infinite Loops 38
Using while 38
Using for 38
13.7 while Loop With a File 39
13.8 while Loop With a CSV File 39
Skipping the header row 40
Chapter 14 — Functions 41
14.1 What Is a Function? 41
14.2 Defining and Calling a Function 41
Style 1 — using the function keyword 41
Style 2 — using parentheses 41
14.3 Passing Arguments to a Function 42
Chapter 15 — Command-Line Arguments 44
15.1 Passing and Accessing Arguments 44
15.2 Special Argument Variables 44

15.3 Looping Over All Arguments 45
15.4 Shifting Arguments 45
The problem shift solves 45
The fix — using shift 46
Chapter 16 — Loop Control and Other Useful Commands 47
16.1 break — Exiting a Loop Early 47
16.2 continue — Skipping to the Next Iteration 47
16.3 The sleep Command 48
16.4 The exit Command 48
16.5 Exit Status 49
Chapter 17 — Mini Project: A Connectivity Check Script 50
17.1 Understanding ping and Exit Status Together 50
17.2 Building the Script 50
Corrected version 50
Chapter 18 — File and Path Utilities 52
18.1 basename — Extract the File Name 52
18.2 dirname — Extract the Directory Path 52
18.3 realpath — Resolve the Full Path 52
18.4 Checking Whether a File or Directory Exists 52
Creating a file if it doesn't exist 53
Chapter 19 — Special Bash Variables 55
19.1 $RANDOM 55
Worked example — a dice roller 55
19.2 $UID 55
Worked example — restricting a script to root only 56
Chapter 20 — Redirection, /dev/null, and Script Identity 57
20.1 What Is Redirection? 57
20.2 Overwrite (>) vs. Append (>>) 57
20.3 Why Redirection Matters for Automation 57
20.4 /dev/null — Discarding Output Entirely 58
20.5 Printing a Script's Own Name — $0 58
Chapter 21 — Logging and Debugging 59
21.1 System Logging With logger 59
21.2 Debugging With set -x 59
21.3 Stopping on the First Error With set -e 60

Chapter 22 — Running Scripts in the Background 61
22.1 The Problem With Running a Script in the Foreground 61
22.2 nohup — Keeping a Script Alive After You Disconnect 61
Chapter 23 — Automating Scripts: at and crontab 62
23.1 One-Time Scheduling With at 62
Scheduling by time 62
Scheduling by date and time together 62
Managing scheduled jobs 62
23.2 Recurring Scheduling With crontab 63
Understanding the Five-Star Format 63
Adding a job with crontab -e 64
Removing a cron job 64
Chapter 24 — Real-World Projects 65
A Note on Approach 65
Project 1 — Monitoring Free RAM Space 65
Step 1 — Explore the free command 65
Step 2 — Filter down to just the number we need 66
Step 3 — Build the full script 66
Project 2 — Monitoring Free Disk Space and Sending an Email Alert 66
Step 1 — Explore the df command 66
Step 2 — Filter down to the filesystem we care about 67
Step 3 — Build the monitoring logic 67
Step 4 — Add an email alert 67
Step 5 — Automate it 68
Project 3 — Archiving Large or Old Log Files 68
Requirement 68
Why This Matters 68
Planning the Steps 69
Building the Script Step by Step 69
Verifying the Script 70
Automating With crontab 71
Project 4 — Automated Local User Creation 71
Requirement 71
Planning the Steps 71
Step 1 — Restrict the Script to Root 72

Step 2 — Require at Least One Argument 72
Step 3 — Capture the Username, and Shift for the Comment 72
Step 4 — Generate a Password 73
Step 5 — Create the User Account 73
Step 6 — Set the Password 73
Step 7 — Force a Password Change on First Login 74
Step 8 — Display the Result 74
The Complete Script 74
Running It 75
Appendix — Quick Reference 76
A.1 Comparison Operators 76
A.2 Loop and Conditional Closing Keywords 76
A.3 Special Variables 76
A.4 File Test Flags 76
A.5 Key Commands Introduced in This Book 77
Closing Notes 77

Chapter 1 — Introduction to Shell Scripting

Shell scripting is one of the most practical skills a person working with Linux or Unix-based systems can learn. Whether you work in software development, system administration, DevOps, cloud computing, or IT support, shell scripts help you automate repetitive tasks, monitor systems, and glue together commands into powerful, reusable tools.

This book is organized as a complete, self-contained course. It begins with the fundamentals — what a shell actually is — and steadily builds up to writing real, production-style automation scripts, including system monitoring tools, email alerting, log archiving, and user account management.

What You Will Learn

- What a shell is, and the different types of shells available on Linux systems
- How to write, save, and execute your first shell script
- Comments, variables, arrays, and string manipulation
- Taking input from users and performing arithmetic
- Conditional logic with if / elif / else and case statements
- Logical operators (AND, OR) for combining conditions
- Loops: for, while, until, and infinite loops
- Functions, and passing arguments to both functions and scripts
- Loop control with break and continue
- Useful built-in commands: sleep, exit, exit status codes
- File and path utilities: basename, dirname, realpath, and existence checks
- Special bash variables such as RANDOM and UID
- Redirection, logging, and debugging techniques
- Running scripts in the background and scheduling them with at and crontab
- Four complete, real-world automation projects

> **NOTE:** *Every concept in this book is followed by the exact worked example used to teach it, so you can type each example yourself and see the same output.*

How This Book Is Organized

Each chapter introduces a concept, explains why it matters, shows the correct syntax, and then walks through a hands-on example — exactly the way you would type it into a terminal. Where the original teaching material built up scripts numbered sequentially (01_basic.sh, 02_add_comments.sh, and so on), this book keeps those same script names so you can follow along in the same order.

By the end of the book, you will have written more than thirty small scripts and four complete real-life automation projects: a RAM monitor, a disk-space monitor with email alerts, a log-archiving tool, and a user-creation utility.

Chapter 2 — Understanding the Shell

2.1 What Is a Shell?

Every computer is built around hardware: the CPU, RAM, hard disk, and other physical components. To make this hardware usable, we install an operating system on top of it — Windows, macOS, or a Linux distribution, for example.

But hardware and the operating system's core (the kernel) cannot be spoken to directly by an ordinary user. We need an interface that accepts our commands and passes them on to the system. This interface is called the shell.

Definition: A shell is a command-line interpreter — a user interface that takes the commands you type, interprets them, and asks the operating system to execute them.

The terminal window you open on a Linux or macOS machine is simply a program that gives you access to a shell.

2.2 Types of Shells

There are several different shell programs available on Unix-like systems, each with slightly different features and syntax. The most common ones are:

Shell	Description
sh	The original Bourne Shell — an older, more basic shell.
bash	"Bourne Again Shell." The most widely used shell today; most people casually call shell scripting "bash scripting" because of how dominant it is.
ksh	The Korn Shell, an enhanced shell that added scripting features on top of sh.
tcsh	An enhanced version of the C shell (csh).
fish	The "Friendly Interactive Shell," designed to be more user-friendly and interactive.
zsh	The Z Shell — an extended version of bash with more advanced features. It is the default shell on modern macOS.

> **NOTE:** *Bash is by far the most commonly used shell in the industry today, which is why this entire book focuses on Bash scripting.*

2.3 Checking Which Shell You Are Using

Linux gives you several ways to inspect your environment before you start scripting.

Check your distribution

View your Linux distribution and version

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ cat /etc/os-release
```

Check your current logged-in user

Displays the currently logged-in user — it is good practice to work as a normal (non-root) user unless a task specifically requires root access.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ whoami
```

Check your default shell

Prints the name of the shell you are currently running, e.g. "bash"

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ echo $0
```

List all shells available on your system

Shows every shell installed on your distribution (typically includes both bash and sh)

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ cat /etc/shells
```

You can change your default shell if you want to, although this book will not go into that process in depth — the key takeaway is simply that Bash is the default and the most widely supported shell, so that is what we will use throughout.

Chapter 3 — What Is Shell Scripting?

A shell script is nothing more than a text file containing a sequence of commands. Instead of typing commands into the terminal one at a time, you write them all into a file once, and then execute that file — all the commands inside run automatically, in order.

Definition: Shell scripting is the practice of writing a set of shell commands into a file so that they can be executed together, repeatedly, and automatically.

3.1 Why Use Shell Scripts?

- Efficiency — instead of running 20 commands manually every time, write them once and reuse the file forever.
- Automation — a script can be scheduled to run automatically at a specific date and time, without you needing to be present at the terminal.
- Consistency — running the same file guarantees the same sequence of steps every time, reducing human error.
- Reusability — once built, a script can be shared, version-controlled, and reused across servers and teams.

> **NOTE:** *If you had to perform the same 20 tasks manually every day, a shell script lets you do all 20 at once — and you can schedule it so it runs even while you are away from the keyboard. This is what we mean by automation.*

Chapter 4 — Writing and Running Your First Script

4.1 Setting Up a Working Folder

Before writing any scripts, it helps to keep them organized in a dedicated folder.

Create and move into a folder to hold all your practice scripts

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ pwd
isha@Ubuntu:~/Desktop/bash-scripting$ ls
isha@Ubuntu:~/Desktop/bash-scripting$ mkdir my_scripts
isha@Ubuntu:~/Desktop/bash-scripting$ cd my_scripts
isha@Ubuntu:~/Desktop/bash-scripting$ pwd
```

4.2 The Shebang Line

The very first line of almost every shell script is a special line called the shebang (a combination of "hash" and "bang," referring to the # and ! characters):

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
```

What it does: The shebang tells the operating system exactly which interpreter should be used to run the rest of the file — in this case, /bin/bash. This matters because your default shell might not always match the shell the script was written for; on some machines the default shell could be an older version, ksh, or something else entirely, which could cause compatibility problems.

> **NOTE:** *The shebang line is not strictly mandatory — a script can technically run without it — but including it is considered a best practice and costs you nothing.*

4.3 Naming Your Script

It is common practice to number scripts as you build them, so you can track what was created and in what order (a habit this book follows too, starting with 01_basic.sh).

Is the .sh extension required? No — a script will run perfectly well without it. However, using .sh is strongly recommended: when another person browses a folder and sees a file ending in .sh, they immediately understand it is a shell script. Without the extension, the purpose of the file is unclear.

4.4 Creating the Script

Scripts can be created using a text editor such as vi (or vim). To create and open a new file for editing:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ vi 01_basic.sh
```

Inside vi, press i to enter Insert mode, then type your script:

01_basic.sh — our first script

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo Alright buddy
```

To save and exit vi: press Escape to leave Insert mode, then type :wq (write and quit) and press Enter.

You can view the contents of any script using the cat command:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ cat 01_basic.sh
```

4.5 Understanding echo

echo sends output to the terminal. Whatever text follows the echo command is printed on screen — this is how a script communicates results, messages, or status back to the person running it.

4.6 Running a Shell Script

Before a script can be executed directly, it needs execute (“x”) permission. You can inspect a file's permissions with:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ls -ltr
```

Permissions are shown in three groups of three: the first set is for the file's owner (user), the second for the group, and the third for everyone else (“others”). A freshly created file typically only has read and write (rw) permission — not execute.

Method 1 — Run the script using the bash command

This method does not require execute permission at all, which makes it the simplest way to get started:

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
isha@Ubuntu:~/Desktop/bash-scripting$ bash 01_basic.sh
```

Method 2 — Run the script directly with ./

If you try to run a script directly without execute permission, you will see a “Permission denied” error:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ./01_basic.sh
# bash: ./01_basic.sh: Permission denied
```

To fix this, grant execute permission using chmod (“change mode”):

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ chmod +x 01_basic.sh
isha@Ubuntu:~/Desktop/bash-scripting$ ls -ltr        # the file name will now
appear in a different colour, confirming it is executable
isha@Ubuntu:~/Desktop/bash-scripting$ ./01_basic.sh
```

What does "./" mean? The dot (.) represents the current directory, and the forward slash (/) separates it from the file name. "./01_basic.sh" therefore means “run the file 01_basic.sh that is located in the directory I am currently in.”

Method 3 — Run a script using its full path

You are not restricted to running a script only from the folder it lives in. From any location, you can execute a script by specifying its complete path:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ cd ~
isha@Ubuntu:~/Desktop/bash-scripting$ /home/paul/my_scripts/01_basic.sh
```

> **NOTE:** *Whichever method you choose — bash <script>, ./<script> (with execute permission), or the full path — the result is the same: every command inside the file runs in order.*

Chapter 5 — Comments

Comments are lines in a script that are not executed. Their entire purpose is to describe what the code does, so that anyone reading the script later — including you, months from now — can quickly understand its purpose.

5.1 Single-Line Comments

A single-line comment starts with a hash symbol (#). Everything after the # on that line is ignored by the shell.

02_add_comments.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "checking comments"
# This is a single line comment
```

Running this script only prints “checking comments” — the comment line has no effect on execution.

5.2 Multi-Line Comments

For longer explanations spanning several lines, you can use a multi-line comment block. It begins with a colon and an opening quote, and must be closed with the exact same marker at the end:

```
isha@Ubuntu: ~/Desktop/bash-scripting
: '
This is multi-line comments
'
```

> **NOTE:** *If you forget to properly close a multi-line comment, the shell will throw an error such as “unexpected end of file — wanted comment,” because it is still waiting for the closing marker. Always make sure a multi-line comment block is properly closed.*

5.3 Why Comments Matter

Comments make a script self-documenting. When someone new reads your script, comments tell them exactly which line does what, without needing to trace through every command.

Chapter 6 — Variables

6.1 What Is a Variable?

A variable is like a labelled container or box in which you can store a piece of information — a number, a word, or a whole sentence. Once stored, you can refer to that value repeatedly throughout your script, and you can change it whenever you need to.

6.2 Declaring a Variable

The syntax for creating a variable is simple: a name, an equals sign, and a value — with no spaces around the equals sign.

03_variable.sh — declaring variables

```
isha@Ubuntu: ~/Desktop/bash-scripting

a=10
name="Prashant"
age=28
```

> **NOTE:** *When storing text (a string) in a variable, it is good practice to wrap the value in double quotes. The script will still work without them, but quoting avoids subtle bugs, especially when a value contains spaces.*

Simply defining a variable produces no visible output — nothing is printed until you explicitly use it.

6.3 Using (Printing) a Variable

To access the value stored inside a variable, prefix its name with a dollar sign ($). This tells the shell, “this is a variable, substitute its value here.”

03_variable.sh — using variables

```
isha@Ubuntu: ~/Desktop/bash-scripting

#!/bin/bash
name="Prashant"
age=28
echo "My name is $name and Age is $age"
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting

My name is Prashant and Age is 28
```

6.4 Changing a Variable's Value

A regular variable's value can be reassigned at any point later in the script:

```
isha@Ubuntu: ~/Desktop/bash-scripting
name="Prashant"
echo "My name is $name"
name="Paul"
echo "My name is $name"
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
My name is Prashant
My name is Paul
```

This demonstrates that once a variable is defined, its value can be changed again later within the same script.

6.5 Storing a Command's Output in a Variable

Instead of hardcoding a value, you can store the result of a command inside a variable. This is done by wrapping the command in $(...):

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ hostname=$(hostname)
echo "Name of this machine is $hostname"
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
Name of this machine is redhat01
```

Whatever a command prints to the terminal — the output of pwd, ls, uname, date, or any other command — can be captured this way and reused later in the script.

6.6 Constant (Read-Only) Variables

Sometimes a value should never change once it is set — for example, a college name, a configuration path, or a fixed limit. To prevent a variable from being modified anywhere else in the script, declare it with the readonly keyword.

04_constant.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
readonly college="metro"
echo "My college name is $college"

# Attempting to change it later:
college="other"
```

If any later line tries to reassign a readonly variable, the shell raises an error and refuses the change:

```
isha@Ubuntu: ~/Desktop/bash-scripting
04_constant.sh: line 8: college: readonly variable
```

Once a variable is declared readonly, its value can never be changed again for the rest of the script's execution.

Chapter 7 — Arrays

7.1 Why Use an Array?

A regular variable stores one value. But what if you need to store ten values, or fifty? Creating fifty separately named variables is neither practical nor manageable. An array solves this problem: it is a single variable capable of holding multiple values, which can be retrieved individually by their position (index), or all together.

7.2 Defining an Array

An array is created by listing values inside parentheses, separated by spaces (not commas, as in many other programming languages):

05_arrays.sh — a mixed-type array

```
isha@Ubuntu: ~/Desktop/bash-scripting
my_array=("one" 1 20 30.5 "Hello" "Hey buddy")
```

Notice that an array can hold different types of values together — whole numbers, decimals, and strings — since shell arrays have no fixed data type.

7.3 Accessing Array Elements

As in most programming languages, array indexing starts at 0. The first element is at index 0, the second at index 1, and so on.

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "Value at third index is ${my_array[3]}"
```

Syntax breakdown: a dollar sign, curly braces, the array name, then square brackets containing the index number.

7.4 Retrieving All Values

To print every value stored in an array at once, use an asterisk (*) inside the square brackets instead of a specific index:

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "All the values in array are ${my_array[*]}"
```

7.5 Finding the Length of an Array

To count how many elements an array contains, combine a hash symbol with the asterisk:

05_arr.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "Length of array is ${#my_array[*]}"
```

This is useful, for example, when validating data — such as checking whether a stored phone number has the expected number of digits.

7.6 Retrieving a Range of Values

You can extract a slice of consecutive elements by specifying a starting index and how many elements to retrieve from that point:

Starting at index 2, retrieve 2 values

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "Values from index 2: ${my_array[@]:2:2}"
```

7.7 Updating (Appending to) an Array

New values can be added to an existing array using +=, followed by the new values in parentheses:

```
isha@Ubuntu: ~/Desktop/bash-scripting
my_array+=("new" 30 40)
echo "Values of new array are ${my_array[*]}"
```

7.8 Associative Arrays (Key-Value Pairs)

A standard (“indexed”) array retrieves values using numeric positions, which becomes hard to remember once you have many elements. An associative array solves this by letting you attach a descriptive key to each value instead.

An associative array must first be declared explicitly using declare -A:

06_key_value.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
declare -A my_array
my_array=( [name]="Prashant" [age]="28" [city]="Paris" )
```

```
echo "Name is ${my_array[name]}"
echo "Age is ${my_array[age]}"
echo "City is ${my_array[city]}"
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
Name is Prashant
Age is 28
City is Paris
```

NOTE: *Associative arrays are far more readable than remembering numeric positions. If you have ten related pieces of information, retrieving "city" by name is much clearer than remembering it lives at index 7.*

Chapter 8 — String Operations

String operations let you inspect and manipulate text stored in a variable — measuring its length, changing its case, replacing parts of it, or extracting a smaller piece from within it. These operations are essential whenever a script processes text pulled from a file, a command, or user input.

07_string_ops.sh — the variable used throughout this chapter

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
my_var="Hey buddy how are you"
```

8.1 Finding the Length of a String

Prefixing a variable name with a hash sign (inside the ${...} syntax) returns the number of characters it contains:

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "Length of the my_var is ${#my_var}"
```

Output: Length of the my_var is 23

8.2 Converting to Upper Case or Lower Case

Bash provides a compact syntax for case conversion using two caret (^) or two comma (,) characters after the variable name:

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "upper case ${my_var^^}"
echo "lower case ${my_var,,}"
```

Syntax	Effect
${var^^}	Converts the entire string to UPPER CASE
${var,,}	Converts the entire string to lower case

8.3 Replacing Part of a String

You can search for a word inside a variable and replace it with another word, using a forward-slash syntax:

```
isha@Ubuntu: ~/Desktop/bash-scripting
new_var=${my_var/buddy/Paul}
echo "new variable is $new_var"
```

Output: new variable is Hey Paul how are you

Syntax: `${variable/search_word/replacement_word}` — this finds the first occurrence of search_word inside the variable and substitutes replacement_word in its place.

8.4 Slicing (Extracting Part of) a String

To pull out a smaller portion of text from within a longer string, use index-based slicing: specify the starting character position and how many characters to extract from there.

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "after slice ${my_var:4:5}"
```

Remembering that indexing starts at 0, character 4 of “Hey buddy how are you” is the start of the word “buddy,” and taking 5 characters from that point extracts exactly that word.

Output: after slice buddy

Syntax: `${variable:start:length}` — start is the index to begin from, and length is how many characters to extract.

Chapter 9 — Getting Input From the User

So far, every value in our scripts has been hardcoded directly into the file. But a script that always prints "Your name is Prashant" is not very useful to anyone else — every user has a different name. To make scripts genuinely interactive, we need a way to ask the user for information while the script is running.

9.1 The read Command

The read command pauses script execution and waits for the user to type something. Whatever they type is stored into the variable name you provide.

08_user_interaction.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
read name
echo "Your name is $name"
```

When this script runs, the terminal appears to "freeze" — it is not stuck, it is simply waiting for the user to type a value and press Enter.

> **NOTE:** *A script that uses read without any explanatory message can confuse a first-time user, who may not understand why the terminal appears to have stopped responding. Always tell the user what you expect them to type.*

9.2 Prompting the User Clearly

There are two common ways to display an instruction before asking for input.

Method 1 — a separate echo statement

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "What is your name?"
read name
```

Method 2 — the -p flag of read (preferred)

The -p flag lets you combine the prompt message and the input request into a single line, with the cursor waiting right after the message instead of jumping to a new line:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ read -p "What is your name? " name
```

NOTE: *Using read -p is generally the cleaner approach: the prompt and the cursor appear on the same line, which feels more natural to the person typing.*

Chapter 10 — Arithmetic Operations

Almost every non-trivial script eventually needs to perform some kind of calculation — counting files, computing memory usage, working with dates, or generating totals. Bash provides more than one way to carry out arithmetic.

10.1 Why You Can't Just Use * or +

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
x=10
y=2
mult=$x*$y
echo "multiply $mult"
```

Perhaps surprisingly, this does not multiply the numbers — Bash treats * and + as plain text by default inside a normal assignment, so the output simply echoes the literal expression rather than calculating it. To actually perform arithmetic, Bash needs to be told explicitly that a mathematical operation is happening.

10.2 Method 1 — the let Command

Prefixing an assignment with let tells Bash to evaluate the right-hand side as an arithmetic expression rather than as plain text:

09_arithmetic_operations.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
x=10
y=2
let mult=$x*$y
echo "multiply $mult"

let sum=$x+$y
echo "sum $sum"
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
multiply 20
sum 12
```

10.3 Method 2 — Double Parentheses $((...))

The second, and generally more flexible, approach is to perform the calculation directly inside double parentheses. This can be used inline, without needing a separate let statement or an intermediate variable.

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "Subtraction is $x-$y"          # WRONG — inside double quotes this just
prints the literal text
echo "Subtraction is $(($x-$y))"     # CORRECT — evaluated as arithmetic
```

The first line above fails because, inside plain double quotes, the shell has no way of knowing that $x-$y should be calculated rather than printed literally. Wrapping the expression in $((...)) tells Bash explicitly to evaluate it as arithmetic before substituting the result.

Output of the corrected line: Subtraction is 8

> **NOTE:** *Rule of thumb: for arithmetic, either prefix the assignment with let, or wrap the expression in double parentheses $((...)). Both approaches are used heavily throughout the rest of this book, including inside loops and conditionals.*

Chapter 11 — Conditional Statements: if, elif, and case

11.1 The if Statement

Conditional statements let a script make decisions: perform one action if a condition is true, and optionally a different action if it is not. This is the foundation of nearly all meaningful automation logic.

Basic syntax

```
isha@Ubuntu: ~/Desktop/bash-scripting
if [[ condition ]]
then
    # commands if condition is true
else
    # commands if condition is false
fi
```

Notice that, unlike the for and while constructs covered later, an if block is closed with fi — simply the word “if” spelled backwards.

Worked example — pass or fail based on marks

if_else.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
read -p "Enter your marks: " marks

if [[ $marks -gt 40 ]]
then
    echo "you are pass"
else
    echo "you are fail"
fi
```

Running with 30 as input prints “you are fail”; running with 41 prints “you are pass.”

11.2 Single vs. Double Square Brackets

You may see conditions written with either [...] (single brackets) or [[...]] (double brackets). Double brackets are an enhanced version that offers more functionality and is generally easier and safer to use, so this book uses them throughout. Single brackets still work and are widely seen in older scripts.

> **NOTE:** *Always leave a space immediately after the opening bracket and immediately before the closing bracket, e.g. [[$marks -gt 40]] — omitting these spaces is a very common source of syntax errors.*

11.3 Comparison Operators

Bash uses short mnemonic codes (rather than symbols like > and <) for numeric comparisons inside test conditions:

Operator	Meaning
-eq	equal to
-ne	not equal to
-gt	greater than
-ge	greater than or equal to
-lt	less than
-le	less than or equal to

> **NOTE:** *For comparing strings (text) rather than numbers, use == instead of -eq. Mixing these up is a very common beginner mistake — using -eq to compare two strings will not behave as expected.*

11.4 The elif Statement — Handling Multiple Conditions

A plain if/else can only distinguish between two outcomes. When you need to check several conditions in sequence — for example, assigning a grade based on a range of scores — use elif ("else if"). You can chain as many elif blocks as you need.

elif_demo.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
read -p "Enter your marks: " marks

if [[ $marks -ge 80 ]]
then
    echo "first division"
elif [[ $marks -ge 60 ]]
then
    echo "second division"
elif [[ $marks -ge 40 ]]
then
    echo "third division"
else
    echo "you are fail"
```

```
fi
```

This script is tested against several inputs, producing the following results:

Marks entered	Result
90	first division
80	first division (greater than or equal to counts as a match)
79	second division
70	second division
50	third division
30	you are fail

> **NOTE:** *Conditions are checked from top to bottom, and the first one that matches wins — the remaining elif and else branches are skipped once a match is found.*

11.5 The case Statement (Switch Case)

Chaining several elif blocks becomes unwieldy once you have many possible conditions to check — the code becomes long, repetitive, and hard to maintain. When you are comparing a single variable against many possible fixed values, the case statement (similar to “switch” in languages such as Java) is a cleaner alternative.

Basic syntax

```
isha@Ubuntu: ~/Desktop/bash-scripting
case $variable in
    pattern1)
        commands
        ;;
    pattern2)
        commands
        ;;
    *)
        commands   # default / catch-all case
        ;;
esac
```

A case block is closed with esac (“case” spelled backwards), just as if is closed with fi. Each branch ends with a double semicolon (;;), which tells the shell where that branch's commands stop.

Worked example — a simple menu

case_demo.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "current location"
echo "Provide an option: A) print date  B) list of scripts  C) current location"
read -p "Choice: " choice

case $choice in
    A)
        date
        ;;
    B)
        ls
        ;;
    C)
        pwd
        ;;
    *)
        echo "please provide a valid input"
        ;;
esac
```

Entering A prints today's date; entering B lists the scripts in the current folder; entering C prints the current working directory; anything else prints the “please provide a valid input” message from the catch-all * branch.

Multiple commands inside one branch

A single case branch is not limited to one command — you can place as many lines as needed before the closing ;;:

```
isha@Ubuntu: ~/Desktop/bash-scripting
A)
    echo "Today's date is"
isha@Ubuntu:~/Desktop/bash-scripting$ date
    echo "ending"
    ;;
```

NOTE: *A case statement's patterns are not limited to single letters — you can match against numbers, full words, or any string value, as long as it matches exactly what you ask the user for.*

Chapter 12 — Logical Operators

Sometimes a single condition is not enough — a decision may depend on two or more conditions being true at once, or on at least one of several conditions being true. Logical operators let you combine multiple conditions inside a single if statement.

Operator	Symbol	Meaning
AND	&&	Both conditions must be true for the overall result to be true.
OR	||	At least one of the conditions must be true for the overall result to be true.
NOT	!	Reverses (negates) the result of a condition.

12.1 The AND Operator (&&)

Both conditions joined by && must be true; if even one is false, the entire expression is false. A classic example: to vote, a person must be both 18 or older AND a citizen of the correct country.

logical.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
read -p "What is your age? " age
read -p "What is your country? " country

if [[ $age -ge 18 && $country == "India" ]]
then
    echo "you can vote"
else
    echo "you can't vote"
fi
```

Age	Country	Result
20	India	you can vote
20	Nepal	you can't vote
15	India	you can't vote
15	Nepal	you can't vote

NOTE: *Remember: -eq is for comparing numbers, but comparing text (strings) — like a country name — requires the double equals sign (==), not -eq.*

12.2 The OR Operator (||)

With ||, only one of the two conditions needs to be true for the overall result to be true. Replacing && with || in the voting example above changes the meaning: now either being 18+ OR being from India is enough to qualify.

```
isha@Ubuntu: ~/Desktop/bash-scripting
if [[ $age -ge 18 || $country == "India" ]]
then
    echo "you can vote"
else
    echo "you can't vote"
fi
```

Age	Country	Result	Why
20	Nepal	you can vote	Age condition alone is true
15	India	you can vote	Country condition alone is true

12.3 Combining AND and OR

Conditions of the form “condition1 AND condition2 OR condition3” evaluate roughly as: if condition1 is true, condition2 is checked next; otherwise condition3 is checked. Understanding operator combinations like this becomes important as your conditional logic grows more complex.

12.4 The Short-Form (Ternary-Style) Condition

For very simple, single-line decisions, Bash allows a compact form that avoids writing a full if/else block:

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
age=15
[[ $age -ge 18 ]] && echo "adult" || echo "minor"
```

Here, if the condition inside [[...]] is true, the command after && runs (“adult” is printed); if the condition is false, the command after || runs instead (“minor” is printed). This is effectively a one-line version of if/else, useful when the action on each branch is short.

Chapter 13 — Loops

A loop lets a script repeat a block of commands automatically, instead of writing the same lines over and over. Whenever you need to run the same task many times — printing numbers, processing every line of a file, or repeating an action until a condition changes — a loop is the right tool.

13.1 The for Loop

Basic syntax

```
isha@Ubuntu: ~/Desktop/bash-scripting
for variable_name in value1 value2 value3
do
    # commands using $variable_name
done
```

A for loop is closed with done, not with a reversed keyword like fi or esac. The loop variable (commonly named i) takes on each listed value in turn, running the block of commands once per value.

Example — looping over numbers

15_for_loop.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
for i in 1 2 3 4 5 6 7 8 9 10
do
    echo "number is $i"
done
```

Example — looping over strings

```
isha@Ubuntu: ~/Desktop/bash-scripting
for name in "Raju" "Shyam" "Baburao"
do
    echo "name is $name"
done
```

Using a range instead of listing every value

Instead of typing out every number, you can use brace-range expansion, which is especially convenient for long sequences:

```
isha@Ubuntu: ~/Desktop/bash-scripting
for i in {1..5}
do
    echo "number is $i"
done
```

13.2 for Loop With a File

One of the most practically useful patterns is reading values line-by-line from a text file and looping over them. This is done by feeding the output of the cat command into the loop:

for_with_file.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# getting values from a file
file="/home/paul/my_scripts/names.txt"

for name in $(cat $file)
do
    echo "name is $name"
done
```

Given a names.txt file containing Raju, Shyam, Baburao, Paul, and Alex (one per line), running this script prints each name in turn. Whenever the contents of names.txt are updated, simply re-running the script automatically picks up the new values — you never need to touch the script itself.

13.3 for Loop With an Array

A for loop can also walk through every element of an array, using its length to control how many times to iterate:

17_for_with_array.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
my_array=("one" 3 "Hello" "Hi")
length=${#my_array[*]}

for (( i=0; i<length; i++ ))
do
    echo "value of array is ${my_array[$i]}"
done
```

> **NOTE:** *Notice the C-style loop header for ((i=0; i<length; i++)) — this is another place where the double-parentheses arithmetic syntax from Chapter 10 is used, this time to control the loop counter directly.*

13.4 The while Loop

A while loop keeps running for as long as its condition remains true. Unlike a for loop, which typically iterates over a fixed list of values, a while loop's number of repetitions depends entirely on when its condition becomes false.

Basic syntax

```
isha@Ubuntu: ~/Desktop/bash-scripting
while [[ condition ]]
do
    # commands
done
```

Example — counting up to a limit

while_demo.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
count=0
number=10

while [[ $count -le $number ]]
do
    echo "value of count variable is $count"
    let count++
done
```

The loop starts with count at 0. On every pass it prints the current value, then increments count by one using let count++. As soon as count exceeds number (becomes 11), the condition $count -le $number becomes false and the loop stops.

13.5 The until Loop

An until loop is the exact opposite of a while loop: it keeps running for as long as its condition is false, and stops as soon as the condition becomes true.

until_loop.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
a=10

until [[ $a -eq 1 ]]
do
    echo "value of a is $a"
    let a--
done
```

Here a starts at 10 and is decremented each pass with let a--. The loop keeps running because a is not yet equal to 1 — and stops the moment it becomes exactly 1.

Loop type	Runs while...
while	the condition remains TRUE
until	the condition remains FALSE (stops once it becomes TRUE)

13.6 Infinite Loops

Some scripts need to run continuously — monitoring a system, printing a heartbeat message, or taking backups at regular intervals. This is achieved with an infinite loop, one whose condition is always true (or that has no ending condition at all).

Using while

infinite.sh (while version)

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
while true
do
    echo "hi buddy"
    sleep 2s
done
```

Using for

infinite (for version) — no initialization, no condition, no increment

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
for (( ; ; ))
do
    echo "hi buddy"
```

```
    sleep 2s
done
```

Both versions behave identically: they print a message every two seconds, forever. To stop an infinite loop that is running in your terminal, press Ctrl + C.

13.7 while Loop With a File

A while loop can also read a file, line by line, using the read command as its condition:

while_with_file.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting

#!/bin/bash
while read my_var
do
    echo "value from file is $my_var"
done < names.txt
```

The file to read from is supplied after done, using input redirection (<) — this feeds the file's contents into the while read loop one line at a time.

13.8 while Loop With a CSV File

Reading a comma-separated values (CSV) file is a very common real-world requirement. Bash handles this by changing the Internal Field Separator (IFS) to a comma for the duration of the read, and by listing one variable name per column.

test.csv — a sample CSV file

```
isha@Ubuntu: ~/Desktop/bash-scripting

id,name,age
01,Paul,20
02,Alex,30
03,Raju,40
```

while_with_csv.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting

#!/bin/bash
while IFS=',' read id name age
do
    echo "id is $id"
    echo "name is $name"
```

```
    echo "age is $age"
done < test.csv
```

This reads test.csv one line at a time, splitting each line on the comma into three variables — id, name, and age — matching the three columns in the file. Note that on the very first pass, the loop also processes the header row (id, name, age) as if it were data.

Skipping the header row

To avoid processing the header line as data, pipe the file through awk first, filtering out row number 1 (NR stands for “number of row”):

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
awk 'NR!=1 {print $0}' test.csv | while IFS=',' read id name age
do
    echo "id is $id"
    echo "name is $name"
    echo "age is $age"
done
```

NOTE: *A good habit when building complex pipelines like this is to first test the individual command (for example, just the awk 'NR!=1 {print $0}' test.csv part) directly on the terminal to confirm it produces the output you expect, before wiring it into a script. This dramatically reduces debugging time.*

Chapter 14 — Functions

14.1 What Is a Function?

A function is a named, reusable block of code. If you find yourself needing to run the same two, three, or four commands repeatedly throughout a script, it is far cleaner to group them into a function once, and then simply call that function by name whenever you need to run them again.

Benefit: Functions reduce repetition, shorten your script, and make it easier to maintain — a change only needs to be made in one place.

14.2 Defining and Calling a Function

There are two equivalent ways to define a function in Bash:

Style 1 — using the function keyword

function_demo.sh (style 1)

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
function welcome_note {
    echo "=================="
    echo "welcome"
    echo "=================="
}

# to call our function:
welcome_note
welcome_note
welcome_note
```

Style 2 — using parentheses

function_demo.sh (style 2)

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
welcome_note() {
    echo "=================="
    echo "welcome"
    echo "=================="
}

welcome_note
```

Both styles behave identically. To invoke (“call”) a function, you simply write its name on its own line — this runs every command contained inside its curly braces.

In the worked example, a three-line welcome banner needed to be printed four times. Rather than copy-pasting the same three echo lines four times, wrapping them in a function called once, twice, or as many times as needed, keeps the script short and clean.

14.3 Passing Arguments to a Function

The two functions above were static — they always printed exactly the same thing. Functions become far more useful when they can behave differently depending on values passed in at call-time. These are called arguments (or parameters).

Inside a function, $1 refers to the first argument passed when it was called, $2 refers to the second, and so on — exactly like the script-level arguments covered in the next chapter.

function_with_args.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
welcome_note() {
    echo "welcome $1"
    echo "age is $2"
}

welcome_note "Raju" 20
welcome_note "Shyam" 30
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
welcome Raju
age is 20
welcome Shyam
age is 30
```

Notice how the same function produces different output each time, simply because different values were supplied as arguments when it was called.

> **NOTE:** *A useful practice exercise: build a small calculator with four functions — addition, subtraction, multiplication, and division — each accepting two numbers as arguments, combined with a case-based menu (A/B/C/D) that asks the user which operation to perform and for both numbers.*

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
add() {
    local number1=$1
    local number2=$2
```

```
    local sum=$((number1 + number2))
    echo "Sum is $sum"
}
```

The local keyword, as used above, restricts a variable's scope to inside the function — useful for avoiding accidental clashes with variables of the same name elsewhere in the script.

Chapter 15 — Command-Line Arguments

Chapter 14 showed how to pass values into a function. This chapter covers passing values into an entire script, directly from the command line, at the moment you execute it.

15.1 Passing and Accessing Arguments

When you run a script and add extra words after its name, those words become arguments that the script can read using $1, $2, $3, and so on.

26_args.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "first argument is $1"
echo "second argument is $2"
```

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ bash 26_args.sh Raju Shyam
```

Output:

```
isha@Ubuntu: ~/Desktop/bash-scripting
first argument is Raju
second argument is Shyam
```

15.2 Special Argument Variables

Variable	Meaning
$1, $2, $3 ...	Individual arguments, in the order they were supplied
$@	All arguments, as a list
$#	The total number of arguments supplied
$0	The name of the script itself

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "number of arguments are $#"
echo "all the arguments are $@"
```

Running bash 26_args.sh Raju Shyam 12 20 produces: number of arguments are 4, and all the arguments are Raju Shyam 12 20.

15.3 Looping Over All Arguments

Combining $@ with a for loop lets you process every argument the user supplied, one at a time — useful for tasks such as backing up several files named on the command line:

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# for loop to access the values from arguments
for file_name in "$@"
do
    echo "copy file $file_name"
done
```

Running this script with several file names as arguments (for example, test.csv and my_file.csv) loops through each one, performing whatever action is written inside the loop — such as taking a backup of each file in turn.

15.4 Shifting Arguments

The shift command removes the first argument from the argument list, causing every remaining argument to move down by one position ($2 becomes the new $1, and so on). This is especially useful when the first argument has a distinct meaning (such as a username) and everything after it should be treated as one combined block of text (such as a free-form description).

The problem shift solves

Consider a user-creation style script that expects a username as the first argument, and a description as the rest:

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "creating user"
echo "username is $1"
echo "description is $2"
```

This works fine for a single-word description (bash script.sh Paul test), but breaks as soon as the description contains multiple words (bash script.sh Alex test user for QA team) — because $2 only captures the second word (“test”), silently dropping the rest, unless the caller remembers to wrap the description in quotes.

The fix — using shift

shift_arg.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "creating user"
username=$1
shift
description="$@"

echo "username is $username"
echo "description is $description"
```

Here, the first argument is captured as the username, then shift removes it from the argument list. Whatever remains — however many words it contains — is captured together as the description via $@, with no need for the user to manually add quotation marks.

> **NOTE:** *shift is particularly valuable whenever a script's first one or two arguments have a fixed, specific meaning, and everything that follows should be treated as one flexible block of free text.*

Chapter 16 — Loop Control and Other Useful Commands

16.1 break — Exiting a Loop Early

The break keyword immediately stops a loop from running any further iterations, as soon as a specified condition is met. Execution continues with whatever comes after the loop — the loop itself does not run again.

28_break.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
# we just need to confirm if a given number is present or not
for i in 1 2 3 4 5 6 7 8 9 10
do
    if [[ $i -eq 6 ]]
    then
        echo "number is found"
        break
    fi
    echo $i
done
```

This loop counts from 1 upward, but the moment it reaches 6, it prints “number is found” and immediately breaks out — numbers 7 through 10 are never even checked.

16.2 continue — Skipping to the Next Iteration

The continue keyword skips the remaining commands in the current pass of the loop and jumps straight to the next iteration, without stopping the loop entirely.

29_continue.sh — printing only odd numbers

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
for i in 1 2 3 4 5 6 7 8 9 10
do
    let r=i%2
    if [[ $r -eq 0 ]]
    then
        continue
    fi
    echo $i
done
```

Here, i % 2 computes the remainder when i is divided by 2 — zero for even numbers, one for odd numbers. Whenever the remainder is 0 (meaning i is even), continue skips the echo line for that pass and jumps straight back to the top of the loop. The result is that only odd numbers (1, 3, 5, 7, 9) get printed.

Keyword	Effect
break	Stops the loop completely; execution resumes after the loop
continue	Skips only the current iteration; the loop keeps running

16.3 The sleep Command

sleep pauses script execution for a specified duration before continuing to the next command. It accepts a number followed by a unit — s for seconds (the default), m for minutes, and so on.

```
isha@Ubuntu: ~/Desktop/bash-scripting
sleep 2s
sleep 1m
```

sleep is often used to give a background process time to finish, to add spacing between repeated actions in a loop, or to control how frequently a monitoring script checks a condition.

16.4 The exit Command

Whereas break and continue only affect the loop they are inside, exit immediately terminates the entire script — nothing after it runs, no matter how many steps remain.

A common use case is validating that a required condition is met at the very start of a script, before any real work begins. For example, a script that is entirely dependent on receiving at least one command-line argument gains nothing by continuing to run if no argument was given:

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
if [[ $# -eq 0 ]]
then
    echo "please provide at least one argument"
    exit
fi

# the rest of the script only runs if an argument was supplied
echo "first argument is $1"
```

Running this script with zero arguments immediately prints the warning message and stops — none of the remaining logic executes.

16.5 Exit Status

Every command you run in the shell — whether typed directly or inside a script — reports back whether it succeeded or failed. This is called its exit status, and it is available immediately afterward in the special variable $?.

Exit status value	Meaning
0	The previous command completed successfully
Any non-zero value (commonly 1)	The previous command failed / was unsuccessful

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ pwd
isha@Ubuntu:~/Desktop/bash-scripting$ echo $?          # prints 0 – pwd succeeded

isha@Ubuntu:~/Desktop/bash-scripting$ cd /root
isha@Ubuntu:~/Desktop/bash-scripting$ echo $?          # prints 1 (or similar) –
permission denied, so it failed
```

Exit status is extremely useful for building dependent logic — for example, only proceeding to step two if step one definitely succeeded. This concept is used directly in the Connectivity Check project later in this book.

> **NOTE:** *A script's own overall success or failure can also be checked from outside it, with echo $? immediately after it finishes running — a script that used exit 1 partway through will report a non-zero (failure) status to whatever called it.*

Chapter 17 — Mini Project: A Connectivity Check Script

This short project brings together user input, the ping command, sleep, and exit status into one small but genuinely useful tool that checks whether a given website is reachable.

17.1 Understanding ping and Exit Status Together

The ping command sends a network packet to a host and reports whether it received a response. The -c1 flag limits it to sending just a single packet (rather than pinging continuously):

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
isha@Ubuntu:~/Desktop/bash-scripting$ ping -c1 www.google.com
isha@Ubuntu:~/Desktop/bash-scripting$ echo $?     # 0 if the connection
succeeded, non-zero if it failed
```

17.2 Building the Script

connectivity_check.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
read -p "which site do you want to check? " site

ping -c1 $site
sleep 5

if [[ $? -ne 0 ]]
then
    echo "unable to connect to $site"
else
    echo "successfully connected to $site"
fi
```

> **NOTE:** *Careful with $? — it always reflects the exit status of the most recently completed command. In the script above, inserting sleep 5 between ping and the if check would cause $? to reflect sleep's success rather than ping's result, silently breaking the logic. Either remove the sleep call, or capture $? into a variable immediately after ping runs, before any other command executes.*

Corrected version

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
#!/bin/bash
read -p "which site do you want to check? " site
```

```
ping -c1 $site
status=$?

if [[ $status -ne 0 ]]
then
    echo "unable to connect to $site"
else
    echo "successfully connected to $site"
fi
```

Testing this script against www.google.com prints “successfully connected,” while testing against a non-existent host such as localhost-8080 (which does not resolve) prints “unable to connect,” correctly reflecting a 100% packet loss result from ping.

Chapter 18 — File and Path Utilities

Real scripts frequently need to work with file paths — extracting just the file name, just the directory, resolving a relative path into an absolute one, or checking whether a file or folder exists before acting on it. This chapter covers the small utility commands that make this possible.

18.1 basename — Extract the File Name

basename strips away the directory portion of a path, leaving only the file name.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ basename /home/paul/my_scripts/test.csv
```

Output: test.csv

18.2 dirname — Extract the Directory Path

dirname does the opposite of basename: it strips the file name, leaving only the directory path.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ dirname /home/paul/my_scripts/test.csv
```

Output: /home/paul/my_scripts

18.3 realpath — Resolve the Full Path

When you are working deep inside a directory and only have a file's short name, realpath expands it into the complete, absolute path from the root of the filesystem.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ realpath test.csv
```

> **NOTE:** *realpath does not require the target to actually exist — passing an arbitrary word still returns the full path it would resolve to at your current location. This is handy any time a script needs to know its own absolute working location.*

18.4 Checking Whether a File or Directory Exists

Before acting on a file or folder, it is good practice to first confirm it actually exists — this avoids a script failing partway through, or worse, operating on the wrong thing.

Test flag	Checks whether...
-f <path>	a regular file exists at that path
-d <path>	a directory exists at that path

file_exist_check.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
file_path="/home/paul/my_scripts/data.txt"

if [[ -f $file_path ]]
then
    echo "file exists"
else
    echo "file not exists"
    exit 1
fi
```

If the file is missing, this script prints “file not exists” and exits with a status of 1 (failure) — any later steps in a larger script that depend on this file will correctly never run.

Creating a file if it doesn't exist

The negated form of the same test (using ! before the flag) is commonly used to create something automatically when it is missing, rather than simply erroring out:

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
file_path="/home/paul/my_scripts/prashant.test"

if [[ ! -f $file_path ]]
then
    echo "creating file now"
    touch $file_path
else
    echo "file exists"
fi
```

The first time this runs, the file does not exist yet, so it is created. Running the exact same script again now finds the file already present, and simply reports “file exists” instead.

NOTE: *You already met -d used this way in Chapter 6, when checking that a target directory exists before proceeding, and in the Archive Project later in this book, when creating an archive folder only if it is not already present.*

Chapter 19 — Special Bash Variables

Bash automatically defines a number of built-in variables that are always available, without you having to declare them. This chapter covers two of the most useful: RANDOM and UID.

19.1 $RANDOM

$RANDOM generates a new random integer between 0 and 32767 every time it is accessed.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ echo $RANDOM
```

Running this repeatedly produces a different number each time. This makes $RANDOM the building block for tasks such as generating random passwords, simulating dice rolls, or randomly selecting an item from a list.

Worked example — a dice roller

To simulate a six-sided die, we need a random whole number between 1 and 6. Taking the remainder of $RANDOM divided by 6 (using the modulus operator, %) always produces a number from 0 to 5; adding 1 shifts that range to 1–6:

dice.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# generating random number between 1 to 6
let number=$RANDOM%6+1
echo "number is $number"
```

Running this script repeatedly produces a different value each time, evenly spread across 1 through 6 — just like rolling a physical die.

19.2 $UID

$UID holds the numeric user ID of whichever user is currently executing the script.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ echo $UID
```

> **NOTE:** *The root user's UID is always exactly 0, regardless of the system. This makes $UID the standard way for a script to verify that it is being run with root (administrator) privileges before performing sensitive actions.*

Worked example — restricting a script to root only

root_user_check.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# checking if the user is root or not
if [[ $UID -eq 0 ]]
then
    echo "user is root"
else
    echo "user is not root"
fi
```

Run as a normal user, this prints “user is not root”; run as root (or via sudo), it prints “user is root.” This exact pattern is used at the very start of the User Management project later in this book, to guarantee that only an administrator can create new user accounts.

Chapter 20 — Redirection, /dev/null, and Script Identity

20.1 What Is Redirection?

By default, the output of any command you run is printed directly to your terminal screen. Redirection lets you send that output somewhere else instead — typically into a file — using the > and >> operators.

20.2 Overwrite (>) vs. Append (>>)

Operator	Behaviour
>	Overwrite: replaces the entire contents of the target file with the new output
>>	Append: adds the new output to the end of the file, keeping everything already there

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ls > all_files.txt          # writes the file list into all_files.txt, overwriting anything already there
isha@Ubuntu:~/Desktop/bash-scripting$ date >> all_files.txt        # appends the current date to the same file, keeping the earlier content
isha@Ubuntu:~/Desktop/bash-scripting$ hostname >> all_files.txt    # appends the hostname too
```

NOTE: *Running a command with a single > a second time will erase whatever was written there previously. If you want to keep accumulating output over multiple runs — such as a log file — always use >> (append), not > (overwrite).*

20.3 Why Redirection Matters for Automation

A script that has been scheduled to run automatically (see Chapter 23) will not have anyone watching the terminal — because there is no terminal session open in the first place. If that script produces useful output (success messages, error details, results), that output needs to go somewhere it can be reviewed later. Redirecting a script's output into a log file solves exactly this problem:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ping -c1 google.com > redirect.log
```

After this runs, nothing appears on screen — but the full output of the ping command can be reviewed at any time afterward simply by opening (or cat-ing) redirect.log.

20.4 /dev/null — Discarding Output Entirely

Sometimes a command's output is neither worth displaying nor worth keeping in a log — you simply do not care about it. Redirecting output to the special file /dev/null discards it completely; nothing is stored anywhere.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ cd /root                # normally prints
a "Permission denied" error
isha@Ubuntu:~/Desktop/bash-scripting$ cd /root &> /dev/null     # the same
command, but its output/error is discarded entirely
```

Applying this to the connectivity check script from Chapter 17, the noisy raw output of the ping command can be silenced while keeping only the clean, human-readable success/failure message:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ping -c1 $site &> /dev/null
```

20.5 Printing a Script's Own Name — $0

As introduced briefly in Chapter 15, the special variable $0 always holds the name of the currently running script. This is particularly useful in logging, where multiple scripts might write to the same log file, and you need to know exactly which script produced each entry.

script_name.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
echo "the name of this script is $0"
```

Running bash script_name.sh prints: the name of this script is script_name.sh

Chapter 21 — Logging and Debugging

21.1 System Logging With logger

Beyond writing your own custom log files (Chapter 20), Bash also provides a built-in logger command that writes entries into the system's central log file, typically found at /var/log/messages.

logger.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
logger "this is log from $0"
```

After running this script, the new entry can be viewed (with appropriate permissions) using:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ sudo less /var/log/messages
```

The entry includes a full timestamp, the hostname, the username, and the message text — including the script name, thanks to $0.

> **NOTE:** *logger is generally intended for system administrators who have access to central system logs. If you are a normal user without that access, maintaining your own dedicated log file (as shown in Chapter 20) is the more practical approach.*

21.2 Debugging With set -x

When a large script misbehaves, it can be difficult to know exactly which line is causing the problem just by looking at the final output. Adding set -x near the top of a script switches on execution tracing: every command is printed to the terminal immediately before it runs, prefixed with a + sign.

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
set -x

if [[ $# -eq 0 ]]
then
    echo "please provide at least one argument"
    exit
fi
echo "first argument is $1"
```

With set -x enabled, running the script with no arguments shows exactly which condition was checked, which branch was taken, and which command actually executed — making it far easier to pinpoint exactly where a script's logic goes wrong.

21.3 Stopping on the First Error With set -e

By default, a script keeps running even after one of its commands fails — later lines will still execute, which is not always what you want. Adding set -e near the top changes this behaviour: as soon as any command exits with a non-zero (failure) status, the entire script stops immediately.

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
set -e

pwd
date
cd root      # this fails — no such directory
hostname     # this line is now never reached
```

Without set -e, all four commands would run, with cd root simply printing an error and the script continuing on to hostname regardless. With set -e in place, the script halts the instant cd root fails — the final hostname line never executes.

> **NOTE:** *set -x and set -e are frequently used together while developing and testing a new script, and can be removed (or left in, depending on your needs) once the script is verified to work correctly.*

Chapter 22 — Running Scripts in the Background

22.1 The Problem With Running a Script in the Foreground

When you run a script normally from a terminal, that terminal session is tied up for as long as the script is running — you cannot use it for anything else until the script finishes. Worse, the script itself is tied to that terminal session: if you close the terminal window (or your SSH connection drops), the script is terminated along with it, even if it was not finished.

This becomes a real problem for any script that needs to run for an extended period, or that needs to keep running even after you disconnect from the server.

22.2 nohup — Keeping a Script Alive After You Disconnect

nohup ("no hang up") runs a script in a way that detaches it from your terminal session, so it keeps running even if that terminal is closed. Combining it with & (which sends the command to run in the background) frees up your terminal immediately, letting you continue working while the script runs behind the scenes.

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ chmod +x 15_for_loop.sh
isha@Ubuntu:~/Desktop/bash-scripting$ nohup ./15_for_loop.sh &
```

Immediately after running this, your terminal prompt returns to you — the script continues executing independently. Since no output file was specified, nohup automatically captures all of the script's output into a file named nohup.out in the current directory, which you can review at any time.

Without nohup &	With nohup &
Terminal is blocked until the script finishes	Terminal is immediately free for other work
Script stops if the terminal/session closes	Script keeps running even after the terminal closes
Output prints directly to the terminal	Output is captured automatically into nohup.out

NOTE: *You can verify this behaviour yourself by opening two or three separate terminal sessions to the same server: start a long-running script with nohup ... & in one, close that terminal entirely, and confirm from a different session that the script is still running.*

Chapter 23 — Automating Scripts: at and crontab

A script only provides real automation once it can run on its own, without a person manually typing the command each time. Linux provides two complementary tools for this: at, for scheduling a script to run once at a specific future time, and crontab, for scheduling a script to run repeatedly on a recurring schedule.

23.1 One-Time Scheduling With at

Use at when you know the exact next time you want a script to run, and you only need it to run that once.

Scheduling by time

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ at 02:58 am
```

After pressing Enter, the terminal accepts one or more commands to run at that time:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ bash /home/paul/my_scripts/34_redirect.sh
# press Ctrl+D to finish entering commands and confirm the job
```

Scheduling by date and time together

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ at 23:00 31 July 2023
```

Managing scheduled jobs

Command	Purpose
atq	List all currently scheduled at jobs, along with their job IDs
atrm <job_id>	Remove (cancel) a previously scheduled job by its ID

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ atq                    # view scheduled jobs
and their IDs
```

```
isha@Ubuntu:~/Desktop/bash-scripting$ atrm 4            # cancel the job with ID
4
```

23.2 Recurring Scheduling With crontab

For anything that needs to repeat — every day, every Sunday, every 15 minutes — crontab is the appropriate tool.

Command	Purpose
crontab -l	List all cron jobs currently scheduled for the current user
crontab -e	Open the crontab file for editing, to add or change a scheduled job

Understanding the Five-Star Format

Every cron job line begins with five fields (each usually shown as a *), followed by the command to execute. Reading from left to right, the five fields represent:

Field position	Represents
1st (minute)	Minute of the hour (0–59)
2nd (hour)	Hour of the day (0–23)
3rd (day of month)	Day of the month (1–31)
4th (month)	Month (1–12)
5th (day of week)	Day of the week (0–6, Sunday to Saturday)

Five plain asterisks (* * * * *) mean “every minute, of every hour, of every day, of every month, of every day of the week” — i.e. run every single minute. Replacing individual fields narrows this down.

```
isha@Ubuntu: ~/Desktop/bash-scripting
30 18 * * 0    /home/paul/my_scripts/34_redirect.sh
# minute=30, hour=18 (i.e. 18:30), every day-of-month, every month, day-of-week=0
(Sunday)
# In plain words: run every Sunday at 18:30
```

> **NOTE:** *Remembering what each of the five positions means can be tricky at first. An excellent free tool for building and verifying cron schedules is crontab.guru — it translates the five-star syntax into a plain-English sentence as you type.*

Adding a job with crontab -e

```
isha@Ubuntu: ~/Desktop/bash-scripting
```

```
isha@Ubuntu:~/Desktop/bash-scripting$ crontab -e
# inside the editor, press i to insert, then add a line such as:
3 16 * * *    cd /home/paul/my_scripts && ./34_redirect.sh
# save and exit exactly as you would in vi (Esc, then :wq)
```

Once saved, the job runs automatically at the scheduled time — in this example, every day at 16:03 — without any further action needed. You can confirm the job ran successfully by checking whether the expected output file (such as a log file created by the script) appears at that time.

Removing a cron job

Run crontab -e again, delete the relevant line, then save and exit as usual.

Chapter 24 — Real-World Projects

Having covered every core building block of shell scripting — variables, arrays, conditionals, loops, functions, arguments, and system commands — this final part of the book applies all of it to four complete, real-life automation projects, exactly as they would be used by a system administrator or DevOps engineer.

A Note on Approach

Before diving into code, every project here follows the same disciplined structure, which is worth adopting for any script you build in the future:

- Understand the requirement clearly — what exactly should the script achieve, before writing a single line.
- Break the requirement into small, ordered steps.
- Test individual commands directly in the terminal first, before wiring them into the script.
- Handle negative cases (missing files, failed commands) explicitly, rather than assuming everything will succeed.
- Add logging so that when the script runs unattended, you can still tell what happened.
- Automate the finished script with crontab so it runs on its own, going forward.

> **NOTE:** *This chapter-opening checklist is itself one of the most valuable things you can take from this book — professional scripts are built by working through requirements and steps methodically, not by improvising directly at the keyboard.*

Project 1 — Monitoring Free RAM Space

Requirement: build a script that checks how much free memory (RAM) is available on the server, and warns us if it drops below an acceptable threshold.

Step 1 — Explore the free command

The free command reports memory usage. Adding -h makes the numbers human-readable (e.g. GB/MB instead of raw byte counts); adding -m forces the output specifically into megabytes, which is easier to compare against a fixed numeric threshold:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ free -m
```

```
isha@Ubuntu: ~/Desktop/bash-scripting
               total        used        free      shared  buff/cache   available
```

```
Mem:           3783        2900         710          45         173         727
Swap:             0           0           0
```

Step 2 — Filter down to just the number we need

Piping the output through grep isolates the Mem: line, and piping again through awk extracts just the fourth column — the free memory figure in megabytes:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ free -m | grep Mem | awk '{print $4}'
```

Step 3 — Build the full script

ram_status.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
free_space=$(free -m | grep Mem | awk '{print $4}')
threshold=500

if [[ $free_space -lt $threshold ]]
then
    echo "warning ram is running low: ${free_space}MB free"
else
    echo "ram space sufficient: ${free_space}MB free"
fi
```

With free_space at roughly 710 MB and a threshold of 500 MB, the script reports “ram space sufficient.” Lowering the threshold above the current free amount (for example, to 800) correctly flips the result to the warning branch instead.

> **NOTE:** *This script demonstrates the full pattern used throughout production monitoring tools: capture a live system metric into a variable, compare it against a fixed threshold, and branch on the result — the same three-step shape reappears in Project 2 for disk space.*

Project 2 — Monitoring Free Disk Space and Sending an Email Alert

Requirement: check how full a given filesystem is, and — if usage crosses a defined danger threshold — automatically send an email alert, so nobody has to manually check disk usage.

Step 1 — Explore the df command

df reports how much space is used and available across the system's mounted filesystems, along with a usage percentage column:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ df
```

Step 2 — Filter down to the filesystem we care about

Real output includes many irrelevant entries (temporary filesystems like tmpfs, and a header row). Filtering with grep -v excludes unwanted lines, while a second grep isolates the one filesystem we actually want to monitor (in this example, the one mounted at /etc):

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ df | grep -v Filesystem | grep -v tmpfs | grep etc | awk '{print $5}' | tr -d '%'
```

This produces a single clean number — the usage percentage of that filesystem, with the % sign stripped off so it can be compared numerically.

Step 3 — Build the monitoring logic

filesystem_status.sh

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
fs=$(df | grep -v Filesystem | grep -v tmpfs | grep etc | awk '{print $5}' | tr -d '%')

if [[ $fs -ge 80 ]]
then
    echo "warning disk space is low: ${fs}%"
else
    echo "all good"
fi
```

Step 4 — Add an email alert

A warning printed to the terminal is only useful if someone is watching. For genuine unattended monitoring, the script should notify someone directly by email when the threshold is crossed, using the mail command.

> **NOTE:** *Sending email from the command line requires a working mail transfer setup (such as Postfix) already configured on the server. Setting up Postfix itself is a one-time system configuration task and is outside the scope of this chapter — the scripting side assumes it is already in place.*

filesystem_status.sh — final version, with email alerting

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
to="admin@example.com"
fs=$(df | grep -v Filesystem | grep -v tmpfs | grep etc | awk '{print $5}' | tr -
d '%')

if [[ $fs -ge 80 ]]
then
    echo "warning disk space is low: ${fs}%"
    echo "warning disk space is low: ${fs}%" | mail -s "Disk Space Alert" $to
else
    echo "all good"
fi
```

Syntax breakdown: `mail -s "Subject line" recipient@example.com` — the -s flag sets the email's subject line, and the final argument is the recipient's address. The text piped in before it becomes the body of the email.

When run with usage above the 80% threshold, this script both prints a warning locally and sends an email to the configured recipient with the subject “Disk Space Alert” and a body reporting the exact usage percentage.

Step 5 — Automate it

Exactly as covered in Chapter 23, this script can be scheduled with crontab to run automatically — for example, every 15 minutes, or once daily. Nobody needs to remember to check disk space manually ever again: if the threshold is crossed, an email arrives automatically.

Project 3 — Archiving Large or Old Log Files

Requirement

In a given directory, find every file larger than a defined size (for example, 20 MB) — or, as a variant, older than a defined number of days (for example, 10 days) — compress each one, and move the compressed copy into an archive folder (creating that folder first if it does not already exist).

Why This Matters

Servers have finite disk space. An application that continuously writes logs — sometimes gigabytes' worth per day — will eventually exhaust available storage if nothing manages it. Rather than deleting

historical data outright, this script compresses large or old files and relocates them into a dedicated archive folder, both freeing up space and keeping a safe, space-efficient backup for the future.

Planning the Steps

- Provide the path of the directory to scan.
- Check that the directory actually exists before doing anything else.
- Create an archive folder inside it, if one is not already present.
- Find all files in that directory larger than the size threshold.
- Compress each matching file.
- Move each compressed file into the archive folder.
- Automate the whole process with a cron job so it runs on its own, daily.

Building the Script Step by Step

Header, versioning, and configuration variables:

archive_project.sh — variables

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# Revision: 001
# Date of creation: <output of the `date` command>

base="/home/paul/tutorial/find_tutorial"   # directory to scan
days=10                                     # age threshold (for an older-files
variant)
depth=1                                     # how deep `find` should search
run=0                                       # used later as a guard condition
```

Confirm the target directory exists before proceeding:

```
isha@Ubuntu: ~/Desktop/bash-scripting
if [[ ! -d $base ]]
then
    echo "directory does not exist: $base"
    exit 1
fi
```

Create the archive subfolder, only if it is not already there:

```
isha@Ubuntu: ~/Desktop/bash-scripting
if [[ ! -d $base/archive ]]
```

```
then
isha@Ubuntu:~/Desktop/bash-scripting$ mkdir $base/archive
fi
```

Find every file larger than 20 MB and loop through the results:

archive_project.sh — the core logic

```
isha@Ubuntu: ~/Desktop/bash-scripting
for i in $(find $base -maxdepth $depth -type f -size +20M)
do
    if [[ $run -eq 0 ]]
    then
        echo "$(date) archiving $i to $base/archive"
isha@Ubuntu:~/Desktop/bash-scripting$ gzip $i || exit 1
isha@Ubuntu:~/Desktop/bash-scripting$ mv ${i}.gz $base/archive
    fi
done
```

Reading this loop: find searches only the top level of $base (thanks to -maxdepth 1) for regular files (-type f) larger than 20 megabytes (-size +20M), and hands the resulting list of file paths to the for loop one at a time via the variable i.

For each matching file, gzip compresses it in place (turning, for example, applog.txt into applog.txt.gz). If gzip fails for any reason, the script exits immediately with a failure status rather than continuing blindly. If it succeeds, mv relocates the freshly compressed .gz file into the archive folder.

> **NOTE:** *A very easy mistake here is writing +20MB instead of +20M in the find command's -size flag — find's size suffixes are single letters (k, M, G), and adding a stray B causes an "invalid size type" error. Always test a command like this directly on the terminal before embedding it in a script, exactly as recommended at the start of this chapter.*

Verifying the Script

Before this script runs, a sample directory might contain several files, two of which exceed 20 MB, and no archive folder yet:

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ ls -lh
# ... 48M  large_log_1.log
# ... 24M  large_log_2.log
# ... 12M  small_log_1.log
# ... 12M  small_log_2.log
```

After running the script, an archive folder appears, containing large_log_1.log.gz and large_log_2.log.gz — compressed, moved, and no longer taking up their original space in the main directory.

Automating With crontab

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ crontab -e
# inside the editor:
5 1 * * *    /home/paul/archive_project.sh
```

This schedules the script to run automatically at 01:05 every night, keeping the directory permanently clean without any manual intervention.

> **NOTE:** *Remember to give the script execute permission (chmod +x archive_project.sh) so that cron is able to run it, and consider adding logging (Chapter 20) so you can confirm, the next morning, exactly which files were archived overnight.*

Project 4 — Automated Local User Creation

Requirement

- The script must be executed with root (or sudo) privileges — otherwise it should exit immediately with an error message and a non-zero exit status.
- The first command-line argument is treated as the username; everything after it is treated as a free-form comment/description.
- A secure password should be auto-generated for the new user — no manual password entry required.
- On success, the script should display the username, the generated password, and the hostname of the server the account was created on.

Planning the Steps

- Check whether the script is being run with super-user privileges; if not, exit with an error.
- Check that at least one argument (the username) was supplied; if not, print usage guidance and exit.
- Store the first argument as the username.
- Shift the arguments, and treat everything that remains as the comment.
- Generate a random password.
- Create the user account with useradd, using the generated password.
- Verify that user creation actually succeeded before continuing.
- Set the account's password.

- Force the user to change their password on first login.
- Display the final username, password, and hostname.

Step 1 — Restrict the Script to Root

As covered in Chapter 19, the root user's UID is always 0. This is the standard, reliable way to confirm the script is being run with sufficient privileges:

create_user.sh — Step 1

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
if [[ $UID -ne 0 ]]
then
    echo "please run with sudo or as root"
    exit 1
fi
```

Step 2 — Require at Least One Argument

If nobody supplies a username, there is nothing meaningful the script can do — so it should guide the user and stop, rather than fail confusingly further down:

create_user.sh — Step 2

```
isha@Ubuntu: ~/Desktop/bash-scripting
if [[ $# -lt 1 ]]
then
    echo "usage: $0 <username> [comment]"
    echo "creates a new user with the given username and optional comment"
    exit 1
fi
```

Step 3 — Capture the Username, and Shift for the Comment

Exactly the pattern introduced in Chapter 15: capture the first argument, shift it away, and treat everything remaining as one combined comment field — so a multi-word description does not need to be manually quoted by whoever runs the script:

create_user.sh — Step 3

```
isha@Ubuntu: ~/Desktop/bash-scripting
username=$1
shift
comment="$@"
```

Step 4 — Generate a Password

A simple but effective approach is to combine the current time down to nanosecond precision, which is highly unlikely to repeat:

create_user.sh — Step 4

```
isha@Ubuntu: ~/Desktop/bash-scripting
password=$(date +%s%N)
```

> **NOTE:** *This is a simple, teaching-friendly approach to generating a unique value. For production systems, consider a dedicated password generator with configurable complexity (length, symbols, mixed case) rather than relying solely on a timestamp.*

Step 5 — Create the User Account

create_user.sh — Step 5

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ useradd -c "$comment" -m $username

if [[ $? -ne 0 ]]
then
    echo "the account could not be created"
    exit 1
fi
```

Here, -c attaches the comment field to the account, and -m ensures a home directory is created for the new user. Checking $? immediately afterward confirms whether useradd actually succeeded — if it did not, the script stops rather than proceeding to set a password on a user that was never created.

> **NOTE:** *Wrap the comment in double quotes ($comment) when passing it to useradd — since it may legitimately contain multiple words, an unquoted variable would be split apart and misinterpreted as several separate arguments, causing an "only one user may be specified" error.*

Step 6 — Set the Password

Piping the password into passwd via its --stdin option avoids interactive prompting, which is essential for a fully automated script:

create_user.sh — Step 6

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ echo $password | passwd --stdin $username
```

Step 7 — Force a Password Change on First Login

Since the generated password is only ever meant to be used once, the account should be configured to require the user to set their own password the first time they log in:

create_user.sh — Step 7 (expire the password immediately, forcing a reset on first login)

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ chage -d 0 $username
```

Step 8 — Display the Result

create_user.sh — Step 8

```
isha@Ubuntu: ~/Desktop/bash-scripting
echo "username: $username"
echo "password: $password"
echo "host: $(hostname)"
```

The Complete Script

create_user.sh — full script

```
isha@Ubuntu: ~/Desktop/bash-scripting
#!/bin/bash
# Revision: 001

# Step 1: must run as root
if [[ $UID -ne 0 ]]
then
    echo "please run with sudo or as root"
    exit 1
fi

# Step 2: require a username
if [[ $# -lt 1 ]]
then
    echo "usage: $0 <username> [comment]"
    exit 1
fi

# Step 3: capture username, shift, capture comment
username=$1
shift
comment="$@"

# Step 4: generate a password
```

```
password=$(date +%s%N)

# Step 5: create the user
useradd -c "$comment" -m $username
if [[ $? -ne 0 ]]
then
    echo "the account could not be created"
    exit 1
fi

# Step 6: set the password
echo $password | passwd --stdin $username

# Step 7: force password change on first login
chage -d 0 $username

# Step 8: report the result
echo "username: $username"
echo "password: $password"
echo "host: $(hostname)"
```

Running It

```
isha@Ubuntu: ~/Desktop/bash-scripting
isha@Ubuntu:~/Desktop/bash-scripting$ sudo ./create_user.sh Alex "account for the
QA team"
```

A successful run displays the new username, its auto-generated password, and the server's hostname — everything the administrator needs to pass along to the new user, all produced from a single command.

> **NOTE:** *Testing this script also demonstrates the value of checking exit status at every step (Chapter 16): an early version that skipped quoting "$comment" failed at the useradd step with "only one user may be specified," and the Step 5 check caught it immediately instead of silently continuing to set a password on a user that did not exist.*

Appendix — Quick Reference

A.1 Comparison Operators

Numeric	String	Meaning
-eq	==	equal to
-ne	!=	not equal to
-gt		greater than
-ge		greater than or equal to
-lt		less than
-le		less than or equal to

A.2 Loop and Conditional Closing Keywords

Opening keyword	Closing keyword
if	fi
case	esac
for / while / until	done

A.3 Special Variables

Variable	Meaning
$0	Name of the currently running script
$1, $2, ...	Positional arguments passed to the script or function
$@	All arguments, as a list
$#	Number of arguments supplied
$?	Exit status of the last executed command (0 = success)
$RANDOM	A random integer between 0 and 32767
$UID	Numeric user ID of the current user (0 = root)

A.4 File Test Flags

Flag	Tests for
-f	a regular file
-d	a directory

A.5 Key Commands Introduced in This Book

Command	Purpose
echo	Print text or variable values to the terminal
read	Accept input typed by the user
let / $(( ))	Perform arithmetic
chmod +x	Grant execute permission to a script
basename / dirname / realpath	Work with file paths
sleep	Pause execution for a set duration
exit	Terminate a script immediately, with an optional status code
set -x / set -e	Enable step-by-step debugging / stop on first error
nohup ... &	Run a script in the background, detached from the terminal
at / crontab	Schedule a script to run once, or on a recurring basis
logger	Write an entry to the system log

Closing Notes

Shell scripting rewards steady, hands-on practice far more than passive reading. Every script in this book was built by testing individual commands on the terminal first, and only then assembling them into a complete file — the same approach used throughout the four real-world projects in Chapter 24. Following that same discipline in your own work is the single most effective habit you can carry forward.

From here, natural next steps include deepening your knowledge of closely related tools frequently used alongside shell scripting — commands such as grep, cut, sed, awk, and find — as well as exploring topics like process management, system service configuration, and more advanced automation frameworks that build on the foundation this book has covered.